

Investigación de los temas de Etapa 2

Gabo de otro grupo nos recomendó investigar un protocolo llamado **BGP**, para el grupal

ITINERARIO

Redes

1. Encontrar vecinos: Dani
2. Propagar rutas: Sebas
3. Enviar info: Jose

Oper

1. Bytes (vivir en bytes): Hermes
2. Memoria física a virtual: Marina

Tema extra (para adelantar)

1. Ir averiguando cómo sucede la conexión final entre todos los equipos



DESARROLLO

Redes

1. Encontrar vecinos

Cada nodo se encarga de enviar periódicamente un mensaje especial por todas las interfaces activas, generalmente utilizando una dirección multicast, no broadcast para no molestar a todos los hosts en la red.

Lo que envía constantemente se llaman paquetes “Hello” con la información del nodo, la red y la máscara de la interfaz y el intervalo de tiempo en el que cada uno de esos paquetes se va a enviar.

Al momento de que un nodo recibe un hello por parte de otro que está en la misma red y los parámetros son compatibles lo agrega a la tabla de vecinos. Para mantener un nodo en la lista de vecinos este tiene que enviar un “keepalive” para que los demás sepan que sigue en línea.

En el caso de que se deje de enviar señales de vida durante un tiempo se considera que ya no está en línea y se descarta de la lista de vecinos. En algunos protocolos como el OSPF, el propio paquete Hello periódico es el keepalive, no son dos mensajes distintos.

En el caso de que elijamos **BGP como protocolo** de comunicación los vecinos no se descubren mediante Hello multicast si no que se tienen que configurar con la IP del otro extremo y abrir una sesión TCP entre los dos, mandando keepalives continuamente para saber que siguen en línea.

Como funciona en este protocolo es que al inicio de la sesión se envía un mensaje llamado "OPEN", aquí se presentan los parámetros como el AS number, el Router ID y el Hold Time antes de empezar con los keepalives. Los keepalive aquí se envían cada 60 segundos.

2. Propagar rutas

1. Recibir información de los vecinos: Con esta información ya que puede comenzar a construir la red
2. Guardar la información recibida en una estructura de datos: Se ocupa una estructura para guardar la información de los nodos y sus conexiones. Puede ser una lista, matriz de adyacencia, etc, esta estructura se define principalmente según el **protocolo de enrutamiento** que se defina y escoja.
3. Propagar la información entre los nodos: Cada nodo comparte con sus vecinos la información de enrutamiento, esto depende del protocolo escogido, pueden ser por:
 - a. **Distance Vector**: propaga destinos conocidos y sus costos.
 - b. **Link State**: propaga información sobre sus vecinos y enlaces.
 - c. **Path Vector**: propaga el destino junto con el camino utilizado para llegar a él.
4. Construir la tabla de enrutamiento: Después de que la información se haya propagado se pueden determinar las rutas hacia los otros destino, entonces cada nodo va a construir su **propia tabla de enrutamiento** con al menos del destino, siguiendo el nodo al que saltar y el costo de la ruta.
 - a. Ejemplo:

Destino	Siguiente salto	Costo
Nodo B	Nodo B	1
Nodo C	Nodo B	2
Nodo D	Nodo C	3

5. Mantener rutas actualizadas: Los nodos se quedan escuchando si hay novedades en la red (se eliminan rutas, nuevas conexiones, etc) en caso de haber un cambio se actualiza la información y si es necesario se vuelve a calcular la tabla de enrutamiento. Para que se mantenga actualizada la red se pueden usar 2 métodos;
 - a. Actualizaciones por evento: Se envía una actualización en caso de que ocurra un cambio en la red para evitar mandar información constantemente en caso que no hayan cambios
 - b. Actualizaciones por tiempo: Cada cierto tiempo los nodos se vuelven a comunicar con su estado como respaldo en caso de que haya información desactualizada o mensajes que se perdieron

Para mantener un orden con las actualizaciones se puede agregar un número de secuencia para saber cuál fue la última actualización recibida

3. Envío de información

El proceso tiene básicamente 3 etapas:

- a. **Creación del paquete:** Contiene los datos a enviar, IP destino e IP origen (Creo que esto es lo que hicimos en la etapa 1). Una recomendación es que el paquete debe de tener un cierto **TTL** (Time To Live) que va decrementando conforme pasa en los routers o nodos para descartar errores, **así se evitan bucles infinitos en la red ya que los routers pueden por errores comenzar a rotarlo sin sentido**, perdiendo la ruta y el paquete carece de un propósito, provocando broadcast storm o congestión en el tráfico de datos. En caso de que este error pase, se debe hacer algo como **ICMP** (Internet Control Message Protocol), lo que hace esto es que el nodo que descartó el paquete una vez muerto el TTL, le **avisa al emisor** que lo descartó.
- b. **Revisión de la red:** El dispositivo compara la IP destino con su propia red, esto con el fin de saber si la IP destino está cerca (localmente) y podemos hacer el envío inmediato por medio de **ARP (Address Resolution Protocol) que conecta la dirección IP con una dirección MAC de la red física local**, o si necesitamos un enlace extra para esto por medio de un envío **default gateway**.
- c. **Consulta de la tabla:** El router recibe el paquete, lee la IP y revisa la tabla de enrutamiento para elegir la mejor interfaz de salida o salto (reenvío a la red adyacente), esta decisión la hace en base al principio de **longest prefix match** (ubica la ruta que mayor coincidencia tenga conforme a la IP).

En caso de que se necesiten nodos intermedios para el envío de paquetes se necesita un **Reenvío salto a salto (hop-by-hop forwarding)** que es la necesidad de que exista un ciclo o repetición de los pasos b) y c) entre los nodos con sus propias tablas de enrutamiento para llegar al destino.

Para la entrega final del paquete (el router conectado a la red del nodo destino), **se hace una resolución ARP** para entregarlo físicamente (capa 2) al receptor.

En la sección anterior (Propagar rutas) en el punto 5 se ahonda sobre la actualización de rutas, esto es crucial para generar las tablas de enrutamiento que servirán de base para esta parte, pues hay **rutas estáticas** (hardcodeadas en la tabla, útiles en ambientes pequeños y controlados) **o implementar un protocolo de enrutamiento dinámico** (RIP, OSPF, EIGRP son algunas opciones para esta configuración) en redes complejas o muy cambiantes en el tiempo, **esto si hay que definirlo para ver como procedemos y en que mecanismo profundizamos.**

Oper

1. Vivir en bytes (memoria byte-addressable)

- Propuesta de formato para la Tabla de Enrutamiento de cada enrutador (Pi)

IP de Destino Exacta	Interfaz de Salida (NIC o Tarjeta de Red)
4 bytes (IPv4)	N bytes (Nombre ASCII de la interfaz de salida)

- **Memoria Byte-Addressable:** Trabajar con un arreglo, donde cada posición guarda **1 solo byte**.
 - **NO se pueden guardar tipos complejos directamente en la memoria virtual simulada** (struct, string, int de golpe). En la memoria virtual solo se escriben y leen bytes puros e independientes.
 - Todo lo que se quiera guardar hay que **desarmarlo en bytes sueltos para meterlo en memoria**.
 - La reconstrucción de lo que esté guardado en memoria **se hace byte por byte para lectura**.
- Para acceder a la memoria se debe usar un módulo o función que simule el funcionamiento de un MMU (Memory Management Unit).
 - El módulo **MMU es el encargado de traducir las direcciones virtuales en físicas** y proteger el acceso a memoria.
 - **Ningún otro módulo puede tocar el arreglo de bytes directamente.**
- **Representación de los nombres de Interfaces:** Como los nombres de interfaces son conjuntos de char y pueden variar de tamaño, **es recomendable acordar un tamaño fijo de bytes para representar cada interfaz** en la memoria byte-addressable.
 - **Si un nombre de alguna interfaz de red es corto** y tiene una longitud menor al tamaño fijo de bytes, **se agregan bytes de relleno** (padding). Para así lograr que el buscador sepa calcular cada cuántos bytes empieza la siguiente fila de la tabla de enrutamiento, por ejemplo.

2. Memoria fisica a virtual

Por que existe la memoria virtual

En una maquina real el programa nunca ve la memoria fisica. Ve un espacio de direcciones que el **sistema operativo** le INVENTA.

Por que hace esto?

1. Aislar procesos entre si
2. Permitir que un proceso use mas memoria de la que ya hay instalada
3. No obligar a un bloque de datos a estar contiguo en la RAM

Quien hace la conversion de direccion virtual a direccion fisica? pieza de HARDWARE: MMU (nosotros lo simulamos en software)

Como se ve una direccion virtual?

En hexadecimal lleva NUMERO DE PAGINA + OFFSET

Ejemplo: 0x1F3A

- offset (byte bajo): 0x3A = 58
- pagina (byte alto): 0x1F = 31

Si fuera un libro seria como en que pagina esta (numero de pagina - 31), en que renglón esta (offset - 58)

Como se traduce de una direccion virtual a una direccion fisica?

1. se divide la direccion en dos trozos. (separar numero de pagina de offset)
 - a. Se divide por el numero de pagina: cociente es el numero de página
 - b. El residuo es el offset
2. Se traduce el numero de pagina (pagina → marco)
 - a. Primero se busca en el TLB (basicamente un forro de la tabla de paginas con las mas frecuentadas) (cache de traducciones)
 - b. Si falla se busca en la tabla de paginas
 - c. Si vuelve a fallar hay un fallo de pagina
3. Armar: el marco que obtuvimos en el paso anterior se pega con el offset original

